

打开 Agent 黑盒： 基于 eBPF 的 Agent 可观测性探索

李昌昊 / 阶跃星辰



Content 目录

01 背景

02 Agent 可观测性概述

03 业界方案对比与 eBPF 方案探索

04 从 Agent 到 Sandbox

05 总结 & QA



Part 背景

AI Agent 给可观测性带来了什么新问题



Agent ≠ 微服务：传统可观测性假设的失效

AI 智能体具有三个典型特征——行为非确定性、执行路径动态、决策需被审计——这决定了现有可观测方案会遇到盲区。

🧠 决策逻辑追踪

需要 " 白盒化 " 思维链

不仅知道 Agent 做了什么，还要知道为什么这么做

缺失后果：无法区分是 Prompt 问题还是模型幻觉

🔧 工具调用审计

完整的输入 / 输出 / 效用记录

捕获 Agent 调用外部 API 的参数、返回结果

缺失后果：Agent 声称 " 已完成 " 但执行了错误或风险命令

💰 成本与性能归因

Token 级与步骤级消耗监控

将延迟和成本精确对应到每一步

缺失后果：无法定位哪个 Agent 的哪一步在大量消耗 Token

🛡️ 安全与合规

内容审计与拦截

检测敏感数据泄露与提示词注入

缺失后果：Agent 可能正在被越狱攻击或泄露数据库密码



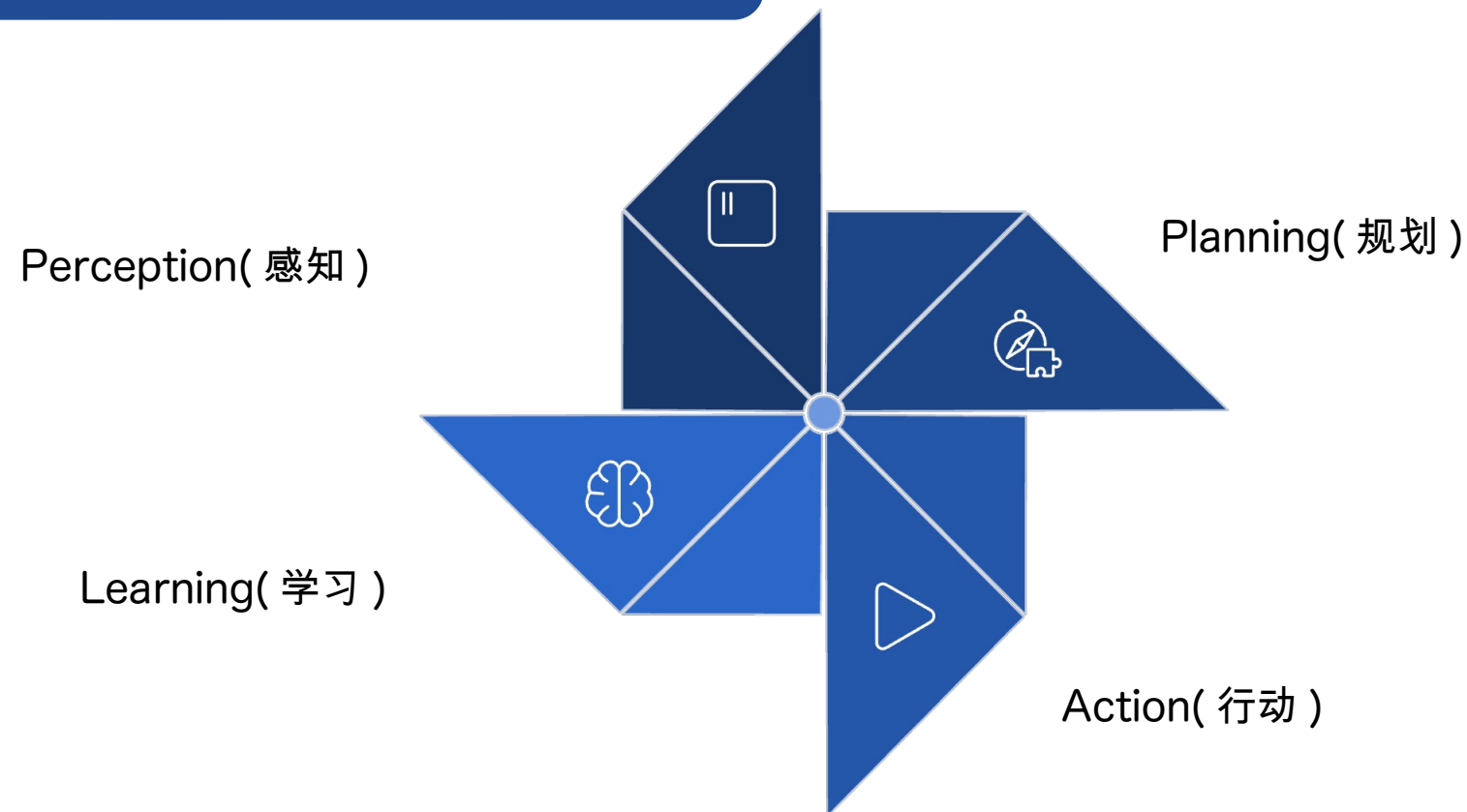
范式转变：从确定性微服务到概率性智能体

传统微服务

- 调用关系：静态定义
- 业务逻辑：确定性代码路径
- 同样的输入 → 同样的输出

AI Agent

- 调用关系：LLM 推理结果动态决定
- 可能自主决定执行敏感操作、被恶意注入



❑ 概率性运行机制 → 当前可观测方案难以满足需求

现有云原生安全可观测体系的盲区

1

盲区① 网络安全 (CNI / Firewall)

✓ 能看到：IP 到 IP 的连接

✗ 看不到：Agent 传输的 Prompt 语义、MCP 协议中的工具调用参数

2

盲区② 审计日志 (K8s Audit)

✓ 能看到：控制平面的 "意图"—— 谁请求了 Exec

✗ 看不到：数据平面的 "实况"—— 容器内部实际执行了什么命令

3

盲区③ 应用性能监控 (APM)

✓ 能看到：HTTP 延迟与错误率

✗ 看不到：Agent 思维链 (CoT) 与实际副作用 (Side Effects) 的关联

❏ Agent 的决策逻辑 (LLM) → 执行行为 (Tool) → 运行结果 (Sandbox) 的完整因果链采集需要新方案。

Part

智能体可观测性概述

介绍智能体可观测性的核心目标、资产模型、交互协议



AI Agent 可观测性核心目标

🎯 意图 (Intent)

Agent 在想什么？

Prompt / Response

Token 用量 / 首 Token 延迟

⚡ 行为 (Action)

Agent 做了什么？

工具调用链 / 命令执行

API 访问 / 文件操作

📊 效果 (Outcome)

结果怎么样？

退出码 / 执行时长

CPU/ 内存消耗 / stdout

🔗 串联 (Trace)

三者如何关联？

完整 Trace Context

将意图→行为→效果串成因果链

🚫 非侵入式观测

不改代码、不加 SDK、不部署中间件

🔍 Agent 行为可溯源

底层系统调用与上层身份、Agent 行为精准关联

📁 资产透明化

自动发现并盘点集群内 Agent 资产及依赖工具链

❏ 定义：智能体可观测性意味着追踪智能体的行为、工具使用情况、模型调用和响应。

Agent 资产模型：Agent / Tool / LLM Source

Agent 系统由三类核心资产构成，彼此通过标准化协议交互。理解这一拓扑结构是设计可观测性方案的基础。

Agent

主动发起请求的智能体工作负载，具备自主决策能力。通过 LLM 推理驱动行为。

A2L : Agent $\rightarrow$ LLM Source

Token 用量 & 推理延迟

Tool

被 Agent 调用的执行单元，包括 kubectl、DB、SaaS API 等。执行具体副作用操作。

A2T : Agent $\rightarrow$ Tool

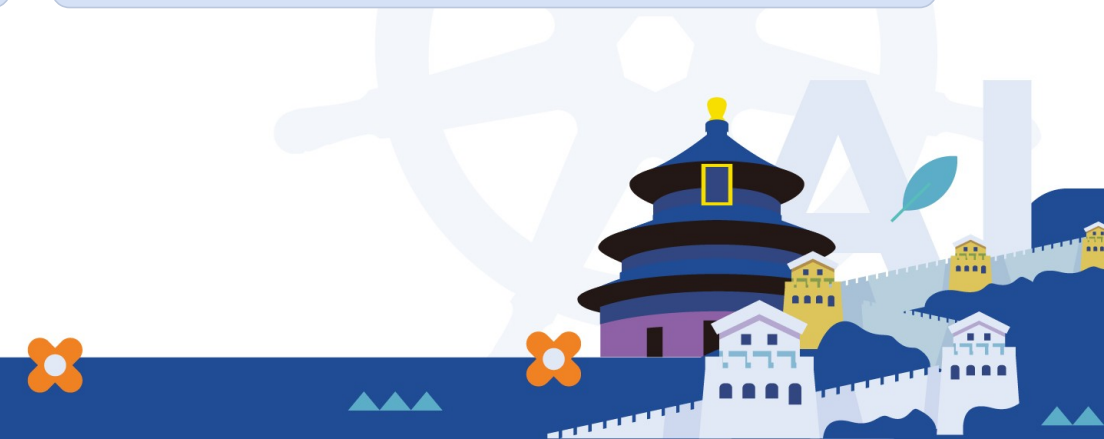
执行结果 & 副作用

LLM Source

提供推理能力的模型端点（OpenAI、Claude、本地模型等），是 Agent 决策的核心驱动力。

A2A : Agent $\rightarrow$ Agent

Trace 传播 & 编排



Agent Infra 生态图谱

K8s 标准层

kubernetes-sigs/agent-sandbox — SIG Apps 正式项目，提供 Sandbox CRD、WarmPool 预热机制与 GKE 深度集成，代表社区方向。

阿里 OpenSandbox

4 类沙箱 (Coding / GUI / Exec / RL)，三级隔离架构，覆盖多样化 Agent 工作负载场景。

OpenKruise AIO Sandbox

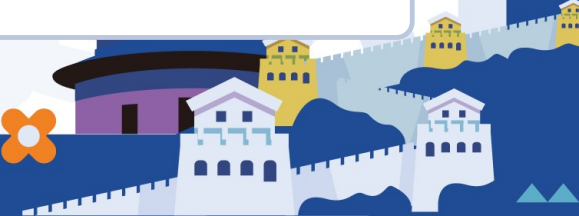
生命周期管理 + E2B 协议兼容 + All-in-One 容器设计，降低多框架接入门槛。

字节 DeerFlow 2.0

子 Agent 编排 + 独立沙箱模式，实现细粒度任务隔离与并发调度。

腾讯云 Agent Runtime

100ms 冷启动，10 万级并发能力，面向高吞吐生产场景优化。



沙箱越安全，观测越困难

沙箱隔离机制在提升安全性的同时，也为可观测性带来了三个根本性挑战：



沙箱即黑盒

隔离做得越好，外部越看不见内部



多沙箱并发

一次 Session 可能操作十几个沙箱实例，需归因到同一决策



生命周期短暂

用完即销，Pull 模式 / Sidecar 来不及采集

❏ 需要一种不依赖沙箱内部配合、能穿透隔离边界、足够轻量的采集方式 → eBPF



业界方案对比与 eBPF 方案探索

从现有方案的盲区出发，深入探索 eBPF 内核级采集的技术选型



现有 Agent 可观测性方案：SDK / 平台 / Proxy

SDK 侵入式

LangSmith / LangFuse / Arize / OpenLLMetry

原理：Agent 代码内嵌 SDK → Callback 拦截 LLM 和 Tool 调用

-  语义最丰富：Prompt / Response / CoT 全可见
-  框架绑定 + 需改代码 + 沙箱运行时完全无感

平台侧采集

百炼 / Dify / Coze

原理：Agent 跑在平台上 → 平台天然记录调用日志

-  零额外接入 + 编排 DAG 完整
-  平台锁定 + 运行时黑盒

Proxy 代理式

LiteLLM / Portkey / Helicone

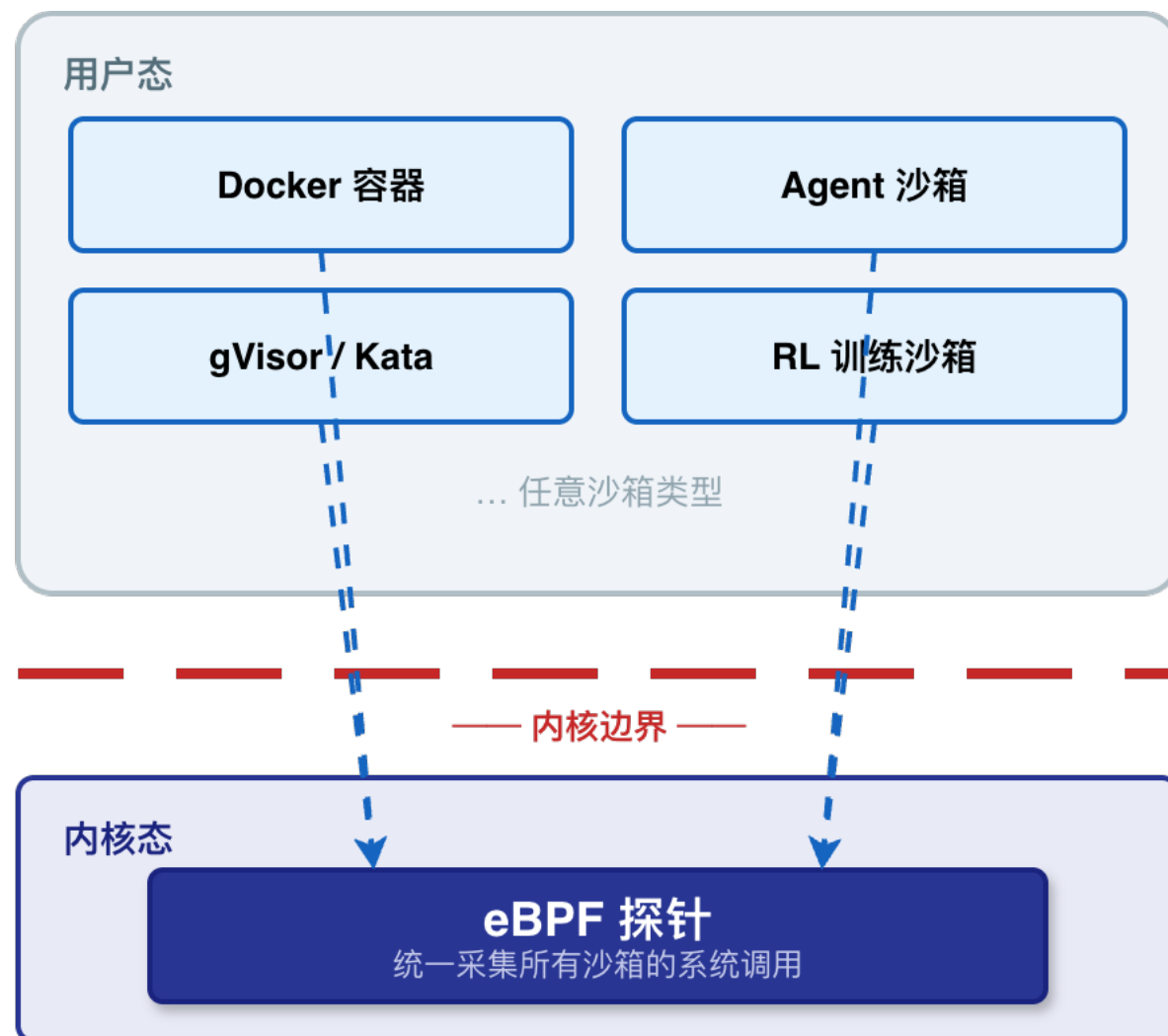
原理：Agent → Proxy 中间人 → LLM API

-  框架无关 + 部署简单
-  只能看 A2L 链路 + 引入延迟和单点 + 无法覆盖 stdio/MCP

❏ 这三类方案有一个共同盲区——沙箱运行时层完全不可见。要穿透这层黑盒，需要把采集点下沉到内核。



eBPF 内核级采集——我们的技术选型



零侵入
不改代码、不加 SDK、不改配置



穿透隔离
namespace 对 eBPF 透明，覆盖所有沙箱类型



全链路覆盖
同时覆盖 LLM 对话 + 工具调用 + 沙箱命令 + MCP

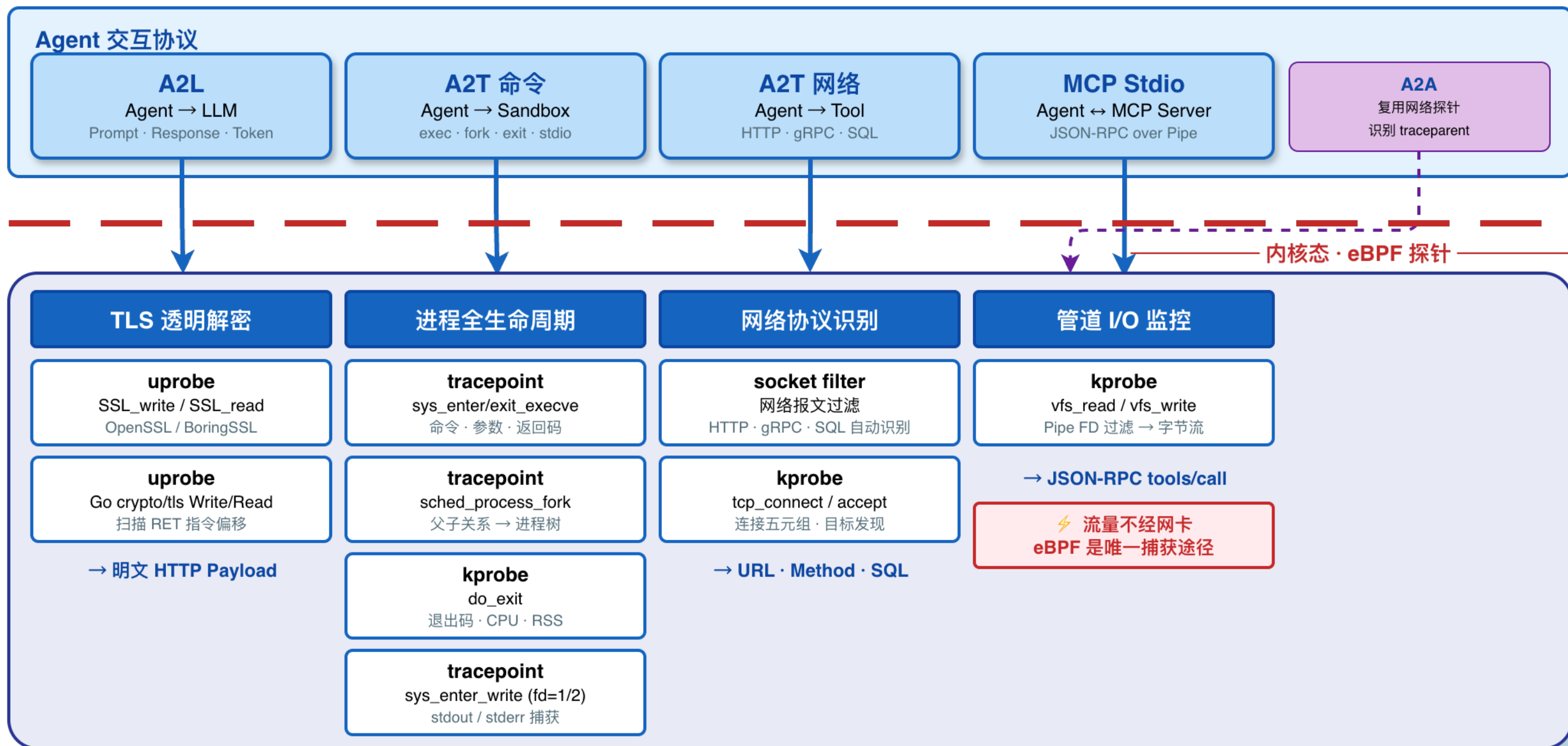


K8s 原生
DaemonSet 部署，cgroup 过滤，热加载



局限性：语义信息需从字节流解析，精度不如 SDK；
对系统环境有要求；

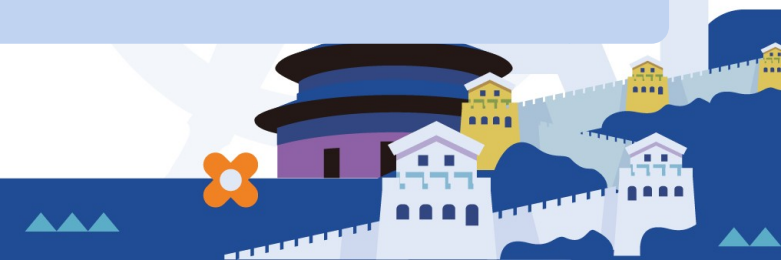
eBPF 采集点全景图



四种方案对比总结：

维度	SDK 侵入	平台侧	Proxy	eBPF
侵入性	🔴 改代码	🔴 绑平台	🟡 改配置	🟢 零
A2L 可见性	🟢 完整	🟢 平台内	🟢 完整	🟢 需解析
A2T 可见性	🟡 部分	🟡 部分	🔴 无	🟢 完整
运行时可见性	🔴 无	🔴 无	🔴 无	🟢 完整
多框架支持	🔴 弱	🔴 弱	🟢 强	🟢 强
语义丰富度	🟢 最高	🟢 高	🟢 高	🟡 中

❏ 方案互补而非互斥 — eBPF 做底层全覆盖，SDK 做上层语义增强

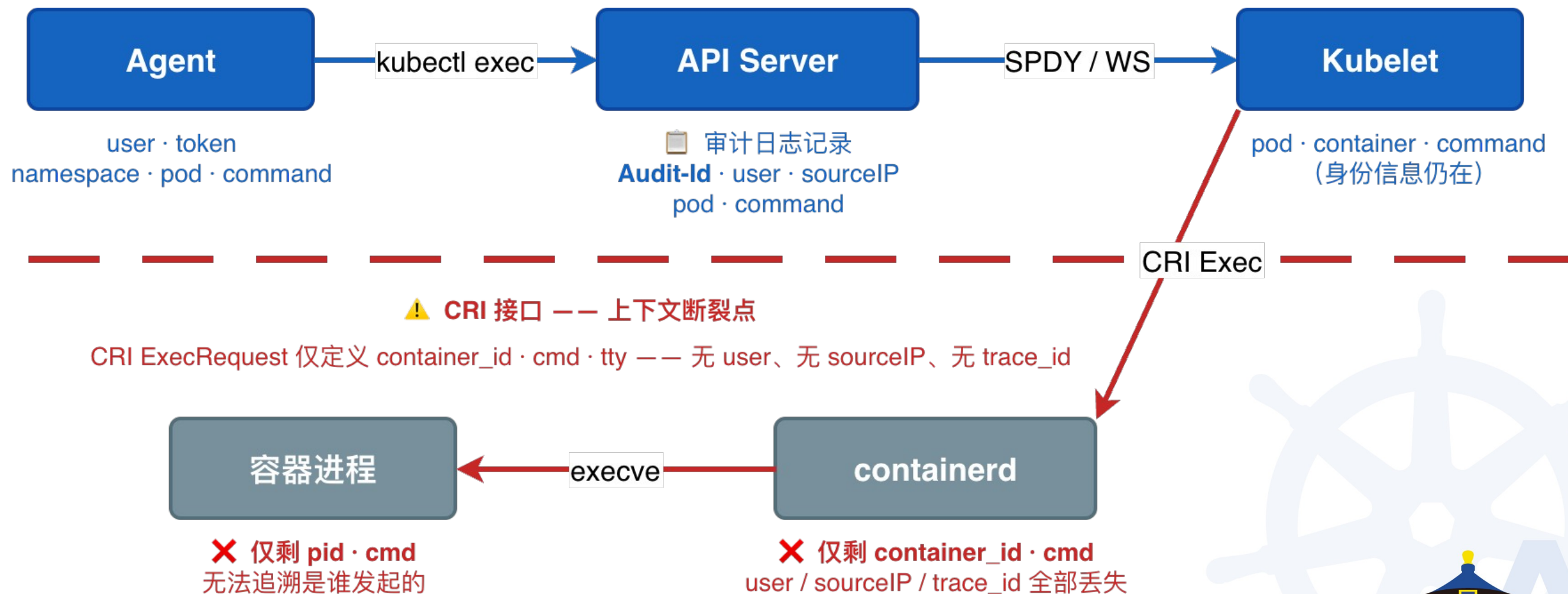


Part 04 Agent 到 Sandbox

容器内执行的命令，能追溯到是哪个 Agent、哪次决策发起的吗？



kubectl exec 链路中的上下文断裂



为什么操作归因很重要

在云原生环境中，单纯依赖审计日志或 eBPF 追踪往往难以还原全貌。操作归因是连接“身份标识”与“行为轨迹”的纽带，解决分布式系统监控中深度可见性的关键缺口。

安全审计

核心问题：执行侧与上下文数据孤立

-  审计日志：记录“谁操作”
-  eBPF：捕获“发生了什么”
-  缺乏归因：供应链攻击与内部威胁不可溯源

Agent 行为追踪

核心问题：因果链路存在断裂

-  短时频繁 exec 调用：难以区分业务逻辑
-  孤立系统调用序列：缺乏上下文关联
-  事故链重建困难：无法定位异常触发点

强化学习信号

核心问题：动作与结果失去匹配

-  优化误导：动作结果无法归因
-  问题感知困难：难以识别运行环境异常

两种方案路径——Agent 注入 & CRI 扩展

方案 A

Agent 侧注入

由执行 `kubectl exec` 的 Agent 在调用前主动将上下文信息写入命令链路，让容器内进程在启动时即可感知 `TRACE_ID` 等关键字段。

- 注入环境变量：`-e TRACE_ID=abc-123`
- Shell 注释携带元数据
- 修改 `argv[0]` 携带标识

方案 B

CRI 层扩展

在 Kubelet 与 containerd 之间的 CRI 协议中新增 `trace_context` 字段，由运行时层面统一完成上下文注入，从根源消除信息丢失。

- 扩展 `ExecRequest` 协议结构
- 新增 `annotations: {trace_id, user}`
- 在调用 `containerd-shim` 之前，`containerd` 统一打印一条包含所有上下文的 Log

❏ A 适合可控的自研 Agent 快速接入，B 是社区正在推进的方向。接下来我们看一种完全不需要任何上游改动的零侵入方案。

方案 C : K8s 审计日志 + eBPF 时空关联 (零侵入) LLM

左流 — K8s Audit Log

- Audit-Id / User / SourceIP
- Pod / Command / Timestamp

右流 — eBPF 运行时事件

- PID / PPID / 命令参数
- 退出码 / Stdio 输出 / Timestamp



 时间窗口匹配

秒级差距



 Pod 身份匹配

PodName ↔ PodUID/cgroup



 命令特征匹配

直接 / basename / shell 包装剥离

❏ 结果 : Audit-Id 绑定到入口进程 → 后续子命令通过 PPID 链继承 → 完整命令序列串联



Agent→Sandbox 操作归因：三方案对比



维度	A：Agent 注入	B：CRI 扩展	C：审计日志 + eBPF
侵入性	● 需 Agent 修改	● 需改 K8s 基础设施	● 零侵入
落地难度	● 中	● 高	● 中
关联精度	● 高（精确 ID）	● 高（原生支持）	● 中高（匹配算法）
适用范围	可控框架	全部（远期）	现有集群即可
可用时间	现在	1-2 年后	现在

☐ 三者互补 — C 做当前基础，A 做可选增强，B 做社区长期方向



Part

总结 & QA



总结

三个关键 Takeaway

1 Agent 可观测性是新领域，
不是传统 APM 的延伸

非确定性执行 + 层级断裂 + 沙箱隔离 → 需要新方案

2 采集方案没有银弹，但
eBPF 在 K8s 环境下有
独特价值

云原生环境下能同时做到零侵入和
全链路覆盖的技术选择

3 操作归因是关键但未标准
化的问题

审计日志 + eBPF 是当前可行方
案 CRI Trace Context 是长期方
向



Thanks.

